

Instructions

Rest App: Run & Manage

Initialization (First Time)

Complete these steps once to get the app ready to run.

Prerequisites

1. Node.js and npm

- Check:

```
node --version  
npm --version
```

- If missing, install LTS from nodejs.org, then reopen your terminal.

2. Docker

- Check:

```
docker --version
```

- If missing, install from docker.com (Mac/Windows) or docs.docker.com (Linux).
- Ensure Docker Desktop/Engine is running.

3. Install Dependencies

- Run once (or after package changes):

```
cd /path/to/femml-casestudy  
npm install
```

4. (Optional) Add environment variables

- The app works out-of-the-box for local dev using default DB credentials and a demo encryption key.
- Create a `.env` file in the project root only if you want to override defaults:

- DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD
- JWT_SECRET
- ENCRYPTION_KEY (64-character hex string)
- Oura live data uses a personal access token entered in the UI, so no Oura env vars are required for local use.

Quick Start (After Initialization)

1. Start everything:

```
npm run start:local
```

This command will:

-  Start the database (PostgreSQL)
-  Set up the database tables
-  Start the backend server (on port 3001)
-  Start the frontend website (on port 5173)

2. (Optional) Add sample data:

```
npm run db:seed
```

This adds 30 days of sample caffeine and sleep logs plus the test user.

3. Wait for startup messages like “VITE ready” and “Server running on port 3001”.

4. Open the app:

```
http://localhost:5173
```

5. Log in (only if you seeded data):

- **Email:** test@example.com
- **Password:** test123

You should now see the app! 🎉

Logging In

- **Test account:** Use test@example.com / test123 after running `npm run db:seed`.
- **Create your own:** Sign up in the app. New accounts start empty.

Additional Test Users

The seed script creates multiple users with different data patterns. The first three have Oura integration enabled:

Email	Password	Oura Connected
test@example.com	test123	✓ Yes
sarah.johnson@demo.com	demo456	✓ Yes
mike.chen@demo.com	demo789	✓ Yes
emma.wilson@demo.com	demo321	✗ No
alex.garcia@demo.com	demo654	✗ No

Each user has 30 days of unique caffeine and sleep data with realistic correlations.

Navigation

The bottom navigation bar has three tabs: -  **Caffeine** - Log your daily caffeine intake
 -  **Sleep** - View and log your sleep data -  **Insights** - See charts and correlations between caffeine and sleep

- You can also go directly to:
 - <http://localhost:5173/caffeine>
 - <http://localhost:5173/sleep>
 - <http://localhost:5173/insights>

Oura Ring Integration & Demo Mode

By default, the app uses demo sleep data. You can switch to live Oura data.

Demo Mode (Default)

- Global toggle:** Top-right “Connect live data to Oura”. When **OFF**, demo data is shown.
- Sleep page toggle:** “Demo: Oura Connected” only shows connected/disconnected UI states.

Connect Live Oura Data

- Toggle “Connect live data to Oura”.
- Paste your Oura token from [Oura Cloud Personal Access Tokens](#).
- Click **Connect Oura Ring**.

What happens: - Stores your token securely - Fetches last 14 days of sleep data - **Overwrites existing Oura sleep logs** with fresh data

Notes: - Caffeine entries are still manual - Oura integration is implemented but **not yet tested with a real ring**



Stopping the App

Press **Ctrl + C** in each terminal running the app.
Stop the database container with:

```
npm run db:stop
```



Start Services Separately

Use this if you want more control or need to troubleshoot.

Step 1: Start the Database

```
npm run db:start
```

Starts PostgreSQL in Docker. Wait for a “container started” message.

Step 2: Set Up Database Tables

```
PGPASSWORD=postgres npm run db:setup
```

Creates tables. Wait for “Database setup complete!”.

Windows note:

```
set PGPASSWORD=postgres && npm run db:setup
```

Step 3: Add Sample Data (Seed)

```
npm run db:seed
```

Adds 30 days of sample caffeine/sleep data.

After seeding, use: - Email: **test@example.com** - Password: **test123**

Step 4: Start the Backend Server

```
npm run dev:server
```

Starts the API server. Wait for “Server running on port 3001”. Keep it running.

Step 5: Start the Frontend (New Terminal)

```
cd /path/to/femml-i-casestudy  
npm run dev
```

Starts the React dev server. Wait for “VITE ready”. Keep it running.

Step 6: Open the App

Go to `http://localhost:5173` in your browser and log in with: - Email: `test@example.com` - Password: `test123`



Deploy to Railway

Railway expects a single web process. This app supports that by serving the built frontend from Express in production.

Build Command

```
npm run build
```

This command will: - Build the frontend (Vite) - Build the backend (TypeScript) - Run database migrations automatically

Start Command

```
npm run start
```

Required Environment Variables

- `DATABASE_URL` (Railway Postgres connection string — use variable reference `{{Postgres.DATABASE_URL}}`)
- `JWT_SECRET` (any long random string, generate with `openssl rand -base64 32`)
- `ENCRYPTION_KEY` (64-character hex string)
- `NODE_ENV=production`

Optional Environment Variables

- `VITE_API_URL` (only needed if you host API separately; otherwise leave unset)

Database Setup on Railway

Database tables are created automatically during the build step via `npm run db:migrate`. The migrations use `CREATE TABLE IF NOT EXISTS` so they are safe to re-run on each deploy.



Seeding the Database

Use this to add sample data.

```
npm run db:seed
```

What it does: - Creates a test user (if missing) - Generates 30 days of caffeine + sleep data - Shows login credentials

Reset and Reseed

```
npm run db:reset
```

Resets the DB and tables.

Run `npm run db:seed` after to add data.



Verifying Data Privacy & Encryption

Sensitive health data is encrypted (AES-256-GCM). The app decrypts it for you.

Here is how you can verify this privacy feature yourself:

1. In the App (Decrypted)

1. Log in at `http://localhost:5173` as `test@example.com`.
2. Go to **Sleep** or **Insights**.
3. You should see normal values (e.g., “82”).

2. In the Database (Encrypted)

Run:

```
node check-data.js
```

You should see encrypted strings like:

```
c0080b2698a0f6c8...:e2354ffed0e4b5dc...:cd97723d9df1abb...
```

Format: IV:authTag:ciphertext.

3. Why this matters

Only the server can decrypt your data; the DB alone shows gibberish.